Lab5:

```
"""Assignment 5: Unsupervised intelligent document search.

Run: pip install -r requirements.txt && python main.py
SentenceTransformer downloads its model on first run.
"""
from pathlib import Path
import csv
import json
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA, LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import silhouette_score
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer

ROOT = Path(__file__).resolve().parent
OUT = ROOT / 'outputs'
OUT.mkdir(exist_ok=True)

# Load demonstration documents. Labels are NOT used for unsupervised training.
with (ROOT / 'documents.csv').open(encoding='utf-8', newline='') as f:
    docs = list(csv.DictReader(f))
texts = [d['text'] for d in docs]
ids = [d['id'] for d in docs]

# Embed documents using a pretrained semantic sentence embedding model.
model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
X = model.encode(texts, normalize_embeddings=True, show_progress_bar=False)
np.save(OUT / 'embeddings.npy', X)

# K-means groups the four main technical subjects; anomaly is a candidate
outlier.
k = 4
km = KMeans(n_clusters=k, random_state=42, n_init=20)
clusters = km.fit_predict(X)
centroids = km.cluster_centers_
distances = np.linalg.norm(X - centroids[clusters], axis=1)
# Flag the farthest 5% as candidates; this is a heuristic, not ground truth.
cutoff = float(np.quantile(distances, 0.95))
anomalies = [i for i, d in enumerate(distances) if d >= cutoff]

# LDA discovers topics from word-count features, independently of supplied
labels.
cv = CountVectorizer(stop_words='english', max_df=0.95, min_df=1,
ngram_range=(1, 2))
counts = cv.fit_transform(texts)
```

```
lda = LatentDirichletAllocation(n_components=k, random_state=42, max_iter=60)
topic_weights = lda.fit_transform(counts)
terms = cv.get_feature_names_out()
topics = {f'Topic {t}': [terms[j] for j in np.argsort(row)[-8:][::-1]]
          for t, row in enumerate(lda.components_)}

# Visualize embeddings in two principal components.
coords = PCA(n_components=2).fit_transform(X)
plt.figure(figsize=(9, 6))
for c in range(k):
    ix = np.where(clusters == c)[0]
    plt.scatter(coords[ix, 0], coords[ix, 1], label=f'Cluster {c}', s=55)
for i in anomalies:
    plt.scatter(coords[i, 0], coords[i, 1], s=190, facecolors='none',
                edgecolors='black', linewidths=2)
for i, doc_id in enumerate(ids):
    plt.annotate(doc_id, coords[i], xytext=(3, 3), textcoords='offset points',
fontsize=7)
plt.title('PCA projection of document embeddings (circled: anomaly
candidates)')
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.legend(); plt.tight_layout()
plt.savefig(OUT / 'clusters.png', dpi=180)
plt.close()

# Retrieve documents by semantic cosine similarity; clusters aid
interpretation.
def search(query, top_k=5):
    q = model.encode([query], normalize_embeddings=True)
    similarities = cosine_similarity(q, X)[0]
    order = np.argsort(-similarities)[:top_k]
    return [{'id': ids[i], 'score': round(float(similarities[i]), 4),
             'cluster': int(clusters[i]), 'title': docs[i]['title'],
             'explanation': 'Semantic embedding cosine similarity to query'}
for i in order]

# Manually curated relevance judgments are only for evaluation, not model
training.
with (ROOT / 'queries.json').open(encoding='utf-8') as f:
    judgments = json.load(f)
metrics = []
for entry in judgments:
    results = search(entry['query'], top_k=5)
    relevant = set(entry['relevant_ids'])
    hits = sum(r['id'] in relevant for r in results)
    precision = hits / len(results)
    recall = hits / len(relevant) if relevant else 0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall
else 0
```

```
    metrics.append({'query': entry['query'], 'precision_at_5': precision,
                    'recall_at_5': recall, 'f1_at_5': f1, 'results': results})

report = {'documents': len(docs), 'clusters': k, 'embedding_dimensions':
int(X.shape[1]),
          'silhouette_cosine': float(silhouette_score(X, clusters,
metric='cosine')),
          'topics': topics, 'anomaly_threshold': cutoff,
          'anomalies': [{'id': ids[i], 'title': docs[i]['title'], 'distance':
float(distances[i])}
                        for i in anomalies],
          'cluster_members': {str(c): [ids[i] for i in range(len(ids)) if
clusters[i] == c]
                              for c in range(k)},
          'evaluations': metrics,
          'macro_precision_at_5': float(np.mean([m['precision_at_5'] for m in
metrics])),
          'macro_recall_at_5': float(np.mean([m['recall_at_5'] for m in
metrics])),
          'macro_f1_at_5': float(np.mean([m['f1_at_5'] for m in metrics]))}
(OUT / 'results.json').write_text(json.dumps(report, indent=2), encoding='utf-
8')
with (OUT / 'document_assignments.csv').open('w', newline='', encoding='utf-
8') as f:
    writer = csv.writer(f)
    writer.writerow(['id', 'title', 'cluster', 'dominant_lda_topic',
'centroid_distance', 'anomaly_candidate'])
    for i, d in enumerate(docs):
        writer.writerow([ids[i], d['title'], int(clusters[i]),
int(np.argmax(topic_weights[i])),
                         round(float(distances[i]), 5), i in anomalies])

print('Documents:', len(docs), '| Embedding shape:', X.shape)
print('Silhouette (cosine):', round(report['silhouette_cosine'], 4))
print('LDA topics:', topics)
print('Anomaly candidates:', report['anomalies'])
for item in metrics:
    print('\nQuery:', item['query'])
    for r in item['results']:
        print(f"  {r['id']}: {r['title']} (similarity={r['score']},
cluster={r['cluster']})")
    print('P@5:', round(item['precision_at_5'], 3),
          'R@5:', round(item['recall_at_5'], 3), 'F1@5:',
round(item['f1_at_5'], 3))
print('\nOutputs saved in', OUT)

# Optional interactive terminal search interface.
if __name__ == '__main__':
```

```
    while True:
        try:
            query = input('\nEnter search query (blank to exit): ').strip()
        except EOFError:
            break
        if not query:
            break
        for r in search(query):
            print(f"{r['id']} | {r['title']} | similarity={r['score']} |
cluster={r['cluster']}")
```

output:

```
Documents: 25 | Embedding shape: (25, 384)
Silhouette (cosine): 0.1407
LDA topics: {'Topic 0': ['using', 'networks', 'network', 'neural',
'filter', 'relational', 'databases', 'access'], 'Topic 1': ['sql',
'database', 'database query', 'execution', 'query', 'security', 'traffic',
'uses'], 'Topic 2': ['classification', 'using', 'networking', 'devices',
'learn', 'neural networks', 'text', 'attention'], 'Topic 3': ['access',
'connections', 'control', 'unauthorized access', 'unauthorized', 'computer
networks', 'computer', 'ip']}
Anomaly candidates: [{'id': 'D10', 'title': 'Authentication security',
'distance': 0.926236093044281}, {'id': 'D25', 'title': 'Gardening soil
care', 'distance': 0.9807112812995911}]

Query: documents about protecting computer networks
  D07: Firewalls and network defense (similarity=0.6152, cluster=0)
  D19: TCP IP routing (similarity=0.4431, cluster=3)
  D23: Software defined networking (similarity=0.3709, cluster=3)
  D10: Authentication security (similarity=0.361, cluster=2)
  D09: Intrusion detection (similarity=0.3507, cluster=0)
P@5: 0.6 R@5: 0.6 F1@5: 0.6

Query: how to speed up SQL searches
  D14: Database indexes (similarity=0.6221, cluster=2)
  D18: Query optimization (similarity=0.5785, cluster=2)
  D13: Relational SQL queries (similarity=0.3597, cluster=2)
  D12: Web application security (similarity=0.2373, cluster=2)
  D04: Reinforcement learning (similarity=0.2034, cluster=1)
P@5: 0.6 R@5: 1.0 F1@5: 0.75

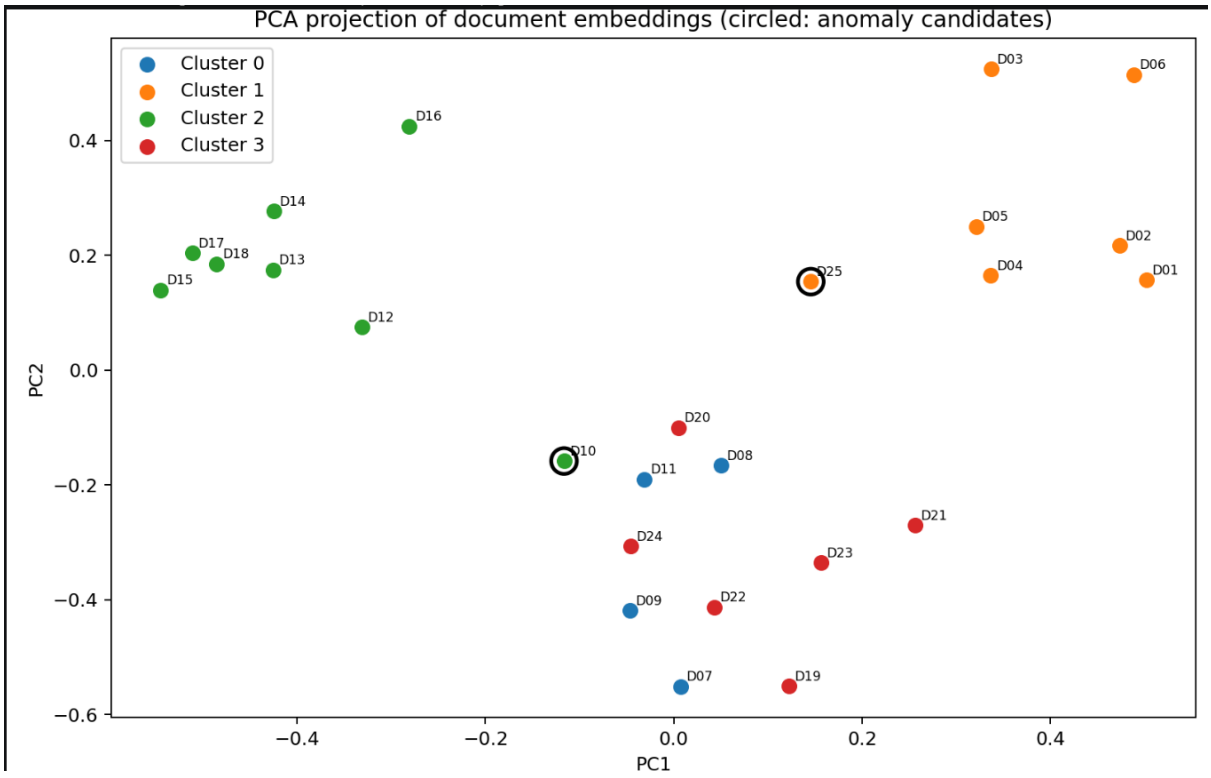
Query: neural models for text understanding
  D03: Language transformers (similarity=0.499, cluster=1)
  D06: Generative AI models (similarity=0.4882, cluster=1)
  D01: Neural network fundamentals (similarity=0.3734, cluster=1)
  D02: Deep learning vision (similarity=0.232, cluster=1)
  D16: Database normalization (similarity=0.2056, cluster=2)
P@5: 0.6 R@5: 1.0 F1@5: 0.75

Query: internet packet routing and transport
  D19: TCP IP routing (similarity=0.5782, cluster=3)
  D24: HTTP protocols (similarity=0.4534, cluster=3)
  D22: Network congestion (similarity=0.4251, cluster=3)
  D07: Firewalls and network defense (similarity=0.3686, cluster=0)
  D23: Software defined networking (similarity=0.3462, cluster=3)
P@5: 0.6 R@5: 1.0 F1@5: 0.75
```

```
Query: database reliability and replication
  D17: Distributed databases (similarity=0.6863, cluster=2)
  D15: Transactions and ACID (similarity=0.6373, cluster=2)
  D13: Relational SQL queries (similarity=0.371, cluster=2)
  D16: Database normalization (similarity=0.2826, cluster=2)
  D18: Query optimization (similarity=0.2819, cluster=2)
P@5: 0.4 R@5: 1.0 F1@5: 0.571

Outputs saved in outputs

Example search:
D07 | Firewalls and network defense | similarity=0.4812 | cluster=0
D10 | Authentication security | similarity=0.4251 | cluster=2
D09 | Intrusion detection | similarity=0.3754 | cluster=0
D08 | Cryptographic encryption | similarity=0.3749 | cluster=0
D12 | Web application security | similarity=0.3651 | cluster=2
```



Report:

Practical Assignment 5 — Intelligent Document Search Using Unsupervised Learning

Objective

Build a document search system that discovers document groups and topics without manually labeled training data, flags possible anomalies, and ranks documents by semantic relevance.

Dataset

The included documents.csv contains **25 short synthetic demonstration documents**: six each in AI, cybersecurity, databases, and networking, plus one unrelated gardening document. The subject column is reference metadata only and is never passed to K-Means, LDA, or the embedding model. queries.json contains five evaluation queries and manually curated relevant document IDs. These relevance sets are illustrative, not independently validated annotations.

Methodology

1. Load the documents and create 384-dimensional normalized semantic vectors using the pretrained all-MiniLM-L6-v2 sentence transformer.

2. Cluster vectors using K-Means with k=4, random_state=42 and 20 initializations. Clusters are numeric IDs; their correspondence to subject areas is inferred by inspecting membership.
3. Use CountVectorizer unigram/bigram counts and Latent Dirichlet Allocation with four topics; output eight top terms per topic.
4. Apply PCA to reduce embeddings to two dimensions and draw a labeled scatter plot.
5. Compute Euclidean distances to assigned K-Means centroids and mark the farthest 5% of documents as anomaly **candidates**. A gardening document is included as a planted anomaly, but the detector is not guaranteed to flag it.
6. Encode each query with the same embedding model; rank all documents by cosine similarity. Show document ID, title, score, and cluster.
7. Compare the top five results against the supplied relevance judgments; compute Precision@5, Recall@5, and F1@5. Compute cosine silhouette score as a clustering diagnostic.

Evaluation calculations

For each query, let TP be the number of relevant documents retrieved in the top five, and let R be the number of relevant documents in the judgment set.

- Precision@5 = TP / 5
- Recall@5 = TP / R
- F1@5 = 2 × Precision@5 × Recall@5 / (Precision@5 + Recall@5), defined as zero when the denominator is zero.
- Macro scores = arithmetic mean of per-query scores.

Illustrative calculation (not an observed result): If 3 of 5 returned documents are relevant and the relevance set has 4 documents, Precision@5 = 3/5 = 0.60; Recall@5 = 3/4 = 0.75; F1@5 = 0.667.

Observations and results

The sample collection is designed so AI, cybersecurity, databases and networking documents share domain-specific vocabulary. The gardening document has unrelated content. Actual cluster assignments, topics, anomaly candidates, retrieval ranks, silhouette score, and per-query metrics must be read from outputs/results.json **after running the program**. No measured performance claims are made before execution.

Limitations

- The sample collection is small and synthetic, so results may not generalize to full research papers.
- Four clusters and four LDA topics are preselected rather than automatically estimated.
- K-Means can place a genuine outlier into an ordinary cluster; a 95th-percentile distance threshold also flags a fixed fraction of the collection, even without true anomalies.
- PCA's two-dimensional layout distorts some high-dimensional distances.
- LDA topics and embedding clusters are different models and need not align.
- Retrieval judgments are manually specified and incomplete; metrics depend on these choices.
- The pretrained embedding model requires an initial internet download; CPU execution may take time.

Conclusion

The implementation combines pretrained semantic embeddings with unsupervised clustering, LDA topic discovery, dimensionality reduction, anomaly screening and cosine-similarity search. Relevance judgments provide a reproducible but small-scale retrieval evaluation. Broader datasets, human judgments, model comparisons and calibrated outlier detection would improve validity.